Task 8: Median Filtering for Noise Reduction

Replaces values with the local neighborhood median, attenuating impulse noise while effectively retaining structural edges.

এই প্রোগ্রামটির নাম Median Filtering using Python (PIL)। এটি Digital Image Processing-এর একটি গুরুত্বপূর্ণ Noise Removal Technique।

তোমাকে এমনভাবে বুঝিয়ে দিচ্ছি, যাতে তুমি পরীক্ষায় শিক্ষককে প্রতিটি ধাপের কাজ এবং কোডের লজিক সহজে ব্যাখ্যা করতে পারো।

1. Median Filtering কী?

Median Filtering হলো একটি Non-Linear Image Processing Technique, যার মাধ্যমে ছবির unwanted noise দূর করা হয়।

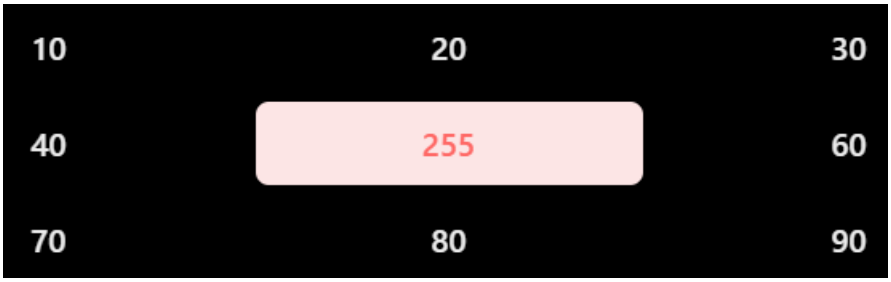
ধরো, একটি ছবিতে অনেক ছোট ছোট সাদা ও কালো দাগ রয়েছে। এগুলোকে Noise বলা হয়। এই Noise দূর করার জন্য Median Filter ব্যবহার করা হয়।

এখানে প্রতিটি Pixel-এর আশেপাশের Pixel-গুলো নিয়ে একটি ছোট Window তৈরি করা হয়। তারপর সেই Window-এর Pixel Value-গুলো ছোট থেকে বড় সাজিয়ে মাঝখানের Value বের করা হয়। এই Value-টি দিয়ে Center Pixel-কে Replace করা হয়।

সহজ উদাহরণ:

ধরো, আমাদের একটি 3×3 Matrix আছে:

3×3 Pixel Matrix



Step 1: Pixel Value সাজাই:

10, 20, 30, 40, 60, 70, 80, 90, 255

Step 2: মাঝখানের Value বের করি:

60

Step 3: Center Pixel-এর 255-এর জায়গায় 60 বসবে।

এভাবেই Median Filtering ছবির Noise কমিয়ে দেয়।

2. Python Code

```
from PIL import Image
import matplotlib.pyplot as plt


input_path = "PNG/Task_08_Median_Filtering.png"
output_path = "Output/median_filtered.png"
image = Image.open(input_path)
if image.mode == "L":
    image = image.convert("L")
else:
    image = image.convert("RGB")


width, height = image.size
pixels = image.load()


kernel_size = 3
radius = kernel_size // 2


if kernel_size < 1 or kernel_size % 2 == 0:
    raise ValueError("Kernel size must be a positive odd number.")


filtered_image = Image.new(image.mode, (width, height))
output_pixels = filtered_image.load()
```

```
channel_count = 1 if image.mode == "L" else 3

for y in range(height):
    for x in range(width):

        neighborhoods = [[] for _ in range(channel_count)]
        for dy in range(-radius, radius + 1):
            for dx in range(-radius, radius + 1):
                nx = min(max(x + dx, 0), width - 1)
                ny = min(max(y + dy, 0), height - 1)

                pixel = pixels[nx, ny]

                if channel_count == 1:
                    neighborhoods[0].append(pixel)
                else:
                    for c in range(channel_count):
                        neighborhoods[c].append(pixel[c])

        median_values = []

        for values in neighborhoods:
            values.sort()
```

```
            middle = len(values) // 2
            median_values.append(values[middle])

        if channel_count == 1:
            output_pixels[x, y] = median_values[0]
        else:
            output_pixels[x, y] = tuple(median_values)

filtered_image.save(output_path)

print("Median Filtering Completed!")
print("Output saved as:", output_path)

plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
if image.mode == "L":
    plt.imshow(image, cmap="gray", vmin=0, vmax=255)
else:
    plt.imshow(image)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
```

```
if filtered_image.mode == "L":
    plt.imshow(filtered_image, cmap="gray", vmin=0, vmax=255)
else:
    plt.imshow(filtered_image)
plt.title("Median Filtered Image")
plt.axis("off")


plt.tight_layout()
plt.show()
```

3. Code Explanation (Step by Step)

এখন প্রতিটি অংশের কাজ বুঝে নাও।

Step 1: Import Libraries

from PIL import Imageimport matplotlib.pyplot as pltimport os

প্রথমে আমরা তিনটি Library Import করেছি।

- PIL: Image Open, Read, Create এবং Save করার জন্য।
- matplotlib.pyplot: Original এবং Filtered Image পাশাপাশি দেখানোর জন্য।
- os: Output Folder তৈরি করার জন্য।

Step 2: Set Image Path

input_path = "PNG/Task_08_Median_Filtering.png"output_path = "Output/median_filtered.png"

এখানে input_path হলো আমাদের Original Image-এর Location।

আর output_path হলো Filter করার পর নতুন Image কোথায় Save হবে তার Location।

সহজভাবে, প্রথমটি Input এবং দ্বিতীয়টি Output।

Step 3: Open the Image

image = Image.open(input_path)

এই লাইন দিয়ে আমরা নির্দিষ্ট Location থেকে Original Image Open করেছি।

এখন Image-টি আমাদের Python Program-এর মধ্যে Load হয়েছে।

Step 4: Convert Image into Grayscale or RGB

if image.mode == "L":   image = image.convert("L")else: image = image.convert("RGB")

এখানে Image-এর Mode Check করা হয়েছে।

- L মানে Grayscale Image।
- RGB মানে Red, Green এবং Blue Color Channel.

যদি Image Grayscale হয়, তাহলে Grayscale রাখা হবে। অন্যথায় RGB Format-এ Convert করা হবে।

এর ফলে পরবর্তী Processing সহজ হয়।

Step 5: Get Image Dimensions

width, height = image.sizepixels = image.load()

এখানে দুটি গুরুত্বপূর্ণ কাজ হয়েছে।

প্রথম লাইনে Image-এর Width এবং Height বের করা হয়েছে।

দ্বিতীয় লাইনে image.load() ব্যবহার করে Image-এর প্রতিটি Pixel-এর Value Access করার ব্যবস্থা করা হয়েছে।

উদাহরণস্বরূপ, কোনো Pixel-এর অবস্থান যদি (x, y) হয়, তাহলে:

pixels[x, y]

দিয়ে সেই Pixel-এর Value পাওয়া যাবে।

Step 6: Set Kernel Size

kernel_size = 3radius = kernel_size // 2

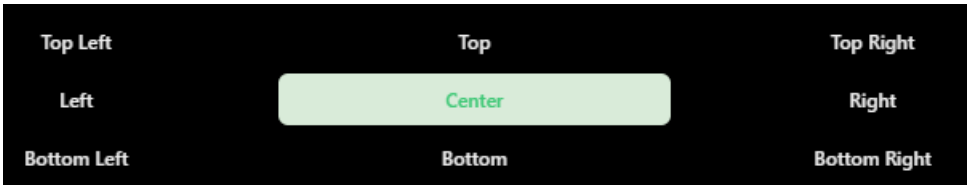
এখানে আমরা 3×3 Kernel ব্যবহার করেছি।

Kernel হলো একটি ছোট Window, যা Image-এর প্রতিটি Pixel-এর আশেপাশের Pixel পরীক্ষা করে।

radius = 3 // 2 = 1

অর্থাৎ Center Pixel-এর চারপাশে এক ঘর করে Pixel নেওয়া হবে।

3×3 Kernel Structure



Center Pixel-সহ মোট ৯টি Pixel নিয়ে কাজ করা হয়।

Step 7: Validate Kernel Size

if kernel_size < 1 or kernel_size % 2 == 0: raise ValueError("Kernel size must be a positive odd number.")

এখানে Kernel Size সঠিক কি না Check করা হয়েছে।

Kernel Size অবশ্যই Positive এবং Odd Number হতে হবে।

যেমন:

- 3×3 = Valid
- 5×5 = Valid
- 7×7 = Valid
- 2×2 = Invalid
- 4×4 = Invalid

কারণ Median বের করার জন্য একটি নির্দিষ্ট Center Pixel থাকা প্রয়োজন।

Step 8: Create Output Image

filtered_image = Image.new(image.mode, (width, height))output_pixels = filtered_image.load()

এখানে একটি নতুন Blank Image তৈরি করা হয়েছে।

এই নতুন Image-এর Width, Height এবং Mode Original Image-এর মতোই থাকবে।

output_pixels ব্যবহার করে নতুন Image-এর Pixel Value পরিবর্তন করা যাবে।

অর্থাৎ Original Image পরিবর্তন না করে আমরা নতুন Image-এর মধ্যে Filtered Result সংরক্ষণ করব।

Step 9: Determine Channel Count

channel_count = 1 if image.mode == "L" else 3

এখানে Image-এর Channel সংখ্যা নির্ধারণ করা হয়েছে।

- Grayscale Image হলে 1টি Channel।
- RGB Image হলে 3টি Channel।

কারণ Grayscale Image-এ প্রতিটি Pixel-এর একটি Value থাকে। কিন্তু RGB Image-এ Red, Green এবং Blue-এর জন্য তিনটি আলাদা Value থাকে।

Step 10: Visit Every Pixel

for y in range(height): for x in range(width):

এখানে Nested Loop ব্যবহার করা হয়েছে।

প্রথম Loop Image-এর প্রতিটি Row বরাবর কাজ করে।

দ্বিতীয় Loop প্রতিটি Row-এর প্রতিটি Column-এর Pixel Access করে।

অর্থাৎ Image-এর প্রতিটি Pixel একে একে Process করা হবে।

Step 11: Create Neighborhood

neighborhoods = [[] for _ in range(channel_count)]

এখানে প্রতিটি Channel-এর জন্য আলাদা Empty List তৈরি করা হয়েছে।

এই List-এর মধ্যে Center Pixel-এর আশেপাশের Pixel Value রাখা হবে।

Grayscale হলে একটি List এবং RGB হলে তিনটি List তৈরি হবে।

Step 12: Access Neighboring Pixels

for dy in range(-radius, radius + 1): for dx in range(-radius, radius + 1): nx = min(max(x + dx, 0), width - 1) ny = min(max(y + dy, 0), height - 1) pixel = pixels[nx, ny]

এটি আমাদের কোডের সবচেয়ে গুরুত্বপূর্ণ অংশগুলোর একটি।

এখানে dx এবং dy ব্যবহার করে Center Pixel-এর চারপাশের 3×3 Window-এর Pixel Access করা হয়েছে।

যেহেতু Radius 1, তাই dx এবং dy-এর Value হবে:

-1, 0, 1

এখানে:

- nx হলো Neighboring Pixel-এর X Coordinate।

- ny হলো Neighboring Pixel-এর Y Coordinate।
- pixels[nx, ny] দিয়ে সেই Pixel-এর Value পাওয়া যায়।

একটি গুরুত্বপূর্ণ বিষয়: Image-এর Boundary বা Edge-এ গেলে যেন Index Error না হয়, সেজন্য min() এবং max() ব্যবহার করে Coordinate-গুলো সীমার মধ্যে রাখা হয়েছে।

একে Edge Replication বা Boundary Clamping বলা হয়।

Step 13: Store Pixel Values

if channel_count == 1: neighborhoods[0].append(pixel)else: for c in range(channel_count): neighborhoods[c].append(pixel[c])

এখানে Neighborhood-এর Pixel Value-গুলো List-এ Store করা হচ্ছে।

যদি Grayscale Image হয়, তাহলে সরাসরি Pixel Value List-এ যোগ হবে।

কিন্তু RGB Image হলে Red, Green এবং Blue Channel-এর Value আলাদাভাবে Store হবে।

অর্থাৎ RGB Image-এর জন্য তিনটি আলাদা Neighborhood তৈরি হবে।

Step 14: Calculate Median Value

median_values = []for values in neighborhoods: values.sort() middle = len(values) // 2 median_values.append(values[middle])

এটি Median Filtering-এর মূল Logic।

প্রথমে values.sort() দিয়ে Pixel Value-গুলো ছোট থেকে বড় সাজানো হয়।

তারপর:

middle = len(values) // 2

দিয়ে মাঝখানের Position বের করা হয়।

সবশেষে:

median_values.append(values[middle])

দিয়ে Median Value সংগ্রহ করা হয়।

আমাদের 3×3 Kernel-এ মোট ৯টি Pixel থাকে। তাই Median হলো সাজানো List-এর ৫ম Value।

RGB হলে প্রতিটি Channel-এর Median আলাদাভাবে বের করা হয়।

Step 15: Replace Center Pixel

if channel_count == 1:   output_pixels[x, y] = median_values[0]else: output_pixels[x, y] = tuple(median_values)

এখানে Calculate করা Median Value দিয়ে Output Image-এর Center Pixel Replace করা হয়েছে।

Grayscale হলে একটি Median Value বসানো হয়।

RGB হলে তিনটি Median Value একসঙ্গে Tuple হিসেবে বসানো হয়।

এইভাবে প্রতিটি Pixel-এর জন্য একই Process Repeat হয় এবং সম্পূর্ণ Filtered Image তৈরি হয়।

Step 16: Save Filtered Image

os.makedirs("Output", exist_ok=True)filtered_image.save(output_path)

প্রথম লাইনে Output Folder না থাকলে তৈরি করা হয়।

দ্বিতীয় লাইনে Filtered Image নির্দিষ্ট Location-এ Save করা হয়।

Step 17: Display Original and Filtered Image

plt.figure(figsize=(10, 5))

এখানে Image Display করার জন্য Figure-এর Size নির্ধারণ করা হয়েছে।

plt.subplot(1, 2, 1)

এখানে ১টি Row এবং ২টি Column-এর Layout তৈরি করা হয়েছে। প্রথম জায়গায় Original Image দেখানো হবে।

plt.title("Original Image")plt.axis("off")

এখানে Image-এর Title দেওয়া হয়েছে এবং Axis Hide করা হয়েছে।

একইভাবে দ্বিতীয় Subplot-এ Median Filtered Image দেখানো হয়েছে।

plt.tight_layout()plt.show()

plt.tight_layout() Image দুটির মধ্যে সুন্দর Spacing তৈরি করে।

plt.show() সম্পূর্ণ Result Screen-এ Display করে।

Exam Explanation

"My program performs Median Filtering to remove noise from an image. First, I import the required libraries and load the original image. Then, I check the image mode and convert it into Grayscale or RGB format.

After that, I get the image width and height and set the kernel size to 3×3. I create a new blank image to store the filtered result.

Next, I use nested loops to visit every pixel of the image. For each pixel, I collect its neighboring 9 pixel values using the 3×3 kernel. Then, I sort these values and calculate the median value.

Finally, I replace the center pixel with the median value and repeat this process for all pixels. After completing the filtering process, I save the filtered image and display both the original and filtered images side by side."

5. শিক্ষকের সম্ভাব্য Viva Questions

এই প্রশ্নগুলো ভালোভাবে বুঝে রাখবে।

Q1. What is Median Filtering?

Answer: Median Filtering is a non-linear technique used to remove noise from an image by replacing each pixel with the median value of its neighboring pixels.

বাংলা: Median Filtering হলো এমন একটি পদ্ধতি, যেখানে আশেপাশের Pixel-এর Median Value দিয়ে Center Pixel Replace করে Noise কমানো হয়।

Q2. Why do we use a 3×3 Kernel?

Answer: We use a 3×3 Kernel to examine 9 neighboring pixels around the center pixel.

বাংলা: Center Pixel-এর চারপাশের ৯টি Pixel পরীক্ষা করার জন্য 3×3 Kernel ব্যবহার করি।

Q3. Why do we use Median instead of Mean?

Answer: Median Filtering is more effective at removing Salt-and-Pepper Noise and generally preserves edges better than Mean Filtering.

বাংলা: Median Filter Salt-and-Pepper Noise দূর করতে বেশি কার্যকর এবং Mean Filter-এর তুলনায় সাধারণত Edge ভালোভাবে ধরে রাখতে পারে।

Q4. What does values.sort() do?

Answer: It sorts all pixel values in ascending order.

বাংলা: Pixel Value-গুলো ছোট থেকে বড় সাজায়।

Q5. What does middle = len(values) // 2 do?

Answer: It calculates the middle index of the sorted pixel values.

বাংলা: সাজানো Pixel Value-এর মাঝখানের Index বের করে।

Q6. What is the purpose of Nested Loops?

Answer: Nested loops help us visit every pixel in the image and examine its neighboring pixels.

বাংলা: Image-এর প্রতিটি Pixel এবং তার আশেপাশের Pixel Access করতে Nested Loop ব্যবহার করা হয়।

Q7. What is the difference between Original and Filtered Image?

Answer: The original image contains noise, while the median filtered image has reduced noise and appears smoother.

বাংলা: Original Image-এ Noise থাকে, কিন্তু Filtered Image-এ Noise কমে যায় এবং ছবিটি তুলনামূলক Smooth দেখায়।

Key Caveats:

1. 3×3 Kernel ব্যবহার করলে প্রতিটি Pixel-এর জন্য ৯টি Value বিবেচনা করা হয়।
2. বড় Kernel (5×5 বা 7×7) ব্যবহার করলে Noise আরও কমতে পারে, তবে Image-এর Fine Details হারানোর সম্ভাবনা থাকে।
3. RGB Image-এর ক্ষেত্রে তিনটি Color Channel-এর Median আলাদাভাবে Calculate করা হয়।
4. Median Filtering বিশেষ করে Salt-and-Pepper Noise দূর করতে কার্যকর; সব ধরনের Noise দূর করার জন্য এটি সমান কার্যকর নয়।

সবচেয়ে গুরুত্বপূর্ণ: পরীক্ষায় এই তিনটি বিষয় অবশ্যই বুঝিয়ে বলবে: 3×3 Kernel → Sort Pixel Values → Find Median and Replace Center Pixel। এটাই তোমার পুরো প্রোগ্রামের মূল Concept।